# Trabajo Práctico: Seguridad e Informática Forense en Bases de Datos

## Equipo 01 - Integrantes


- Tomas Ruiz
- Facundo Sorasio
- Mateo Sparta
- Rodrigo Vargas
- Jose Zerpa


---

## TP 7: Informática Forense de Datos

> **Consigna**
> *Describa el procedimiento para recuperar un registro eliminado físicamente en SQL Server utilizando los logs de transacciones.*

### 1. Introducción y Fundamentos

Cuando se ejecuta un comando `DELETE` en SQL Server, el motor no borra inmediatamente los datos del disco de forma irreversible. En cambio, el proceso queda registrado en el Transaction Log (archivo .ldf), una estructura de escritura secuencial que almacena cada operación realizada sobre la base de datos antes y después de ser aplicada a los datos.

Este mecanismo, conocido como Write-Ahead Logging (WAL), garantiza la atomicidad y durabilidad de las transacciones (propiedades ACID). Desde el punto de vista forense, es precisamente este registro el que permite reconstruir registros eliminados.

### 2. Arquitectura del Transaction Log en SQL Server

#### 2.1 Estructura del archivo .ldf

El Transaction Log está compuesto por Virtual Log Files (VLFs), que son segmentos internos de tamaño variable. Cada VLF contiene una secuencia de Log Records, identificados por un número de secuencia llamado Log Sequence Number (LSN).

| Componente | Descripción |
|---|---|
| **Log Record** | Unidad mínima de registro. Contiene la operación, la tabla afectada y los datos. |
| **LSN (Log Sequence Number)** | Identificador único y secuencial de cada registro en el log. |
| **VLF (Virtual Log File)** | Segmento interno del .ldf. Puede estar activo, inactivo o reutilizable. |
| **MinLSN** | LSN mínimo activo. Define el inicio de la parte del log que no puede ser truncada. |
| **Checkpoint** | Punto de sincronización entre el buffer pool y el disco. Marca VLFs como inactivos. |

### 2.2 Contenido de un registro de operación DELETE

Cuando se ejecuta un DELETE, el motor registra en el log la siguiente información:

- Tipo de operación: LOP_DELETE_ROWS
- Identificadores: nombre de la tabla (object_id), partition_id, slot_id
- Datos anteriores (Before Image): el contenido completo de la fila antes de ser eliminada
- ID de transacción: para correlacionar con BEGIN/COMMIT/ROLLBACK
- Timestamp de la operación
- ID del usuario o login que ejecutó la operación

Es la Before Image el dato crítico para la recuperación forense: contiene los valores de todas las columnas de la fila eliminada tal como existían antes del DELETE.

## 3. Modelos de Recuperación y su Impacto Forense

La disponibilidad del log para análisis forense depende directamente del modelo de recuperación configurado en la base de datos:

| Modelo | Comportamiento del Log | Disponibilidad Forense |
|---|---|---|
| FULL | Registra todas las operaciones. No trunca hasta backup de log. | Alta — ideal para forensia |
| BULK_LOGGED | Operaciones masivas (BCP, BULK INSERT) mínimamente registradas. | Media — parcialmente disponible |
| SIMPLE | El log se trunca automáticamente en cada Checkpoint. | Baja — ventana muy reducida |

Para entornos donde se requiere trazabilidad forense, el modelo FULL es el único recomendado. Permite mantener el historial completo de transacciones entre backups de log.

## 4. Procedimiento de Recuperación Forense Paso a Paso

### Paso 1: Verificar el modelo de recuperación y estado del log

Antes de iniciar el análisis, se debe confirmar que la base de datos utiliza el modelo FULL y que los archivos de log o sus backups están disponibles:

```
-- Verificar modelo de recuperación
SELECT name, recovery_model_desc
FROM sys.databases
WHERE name = 'NombreBaseDatos';
-- Ver VLFs activos y estado del log
DBCC LOGINFO('NombreBaseDatos');
```

### Paso 2: Leer el Transaction Log con fn_dblog()

La función fn_dblog() permite leer los registros del log activo directamente desde T-SQL. Para identificar operaciones de borrado:

```
-- Buscar registros de DELETE en el log activo
SELECT [Current LSN], [Operation], [Transaction ID],
       [Transaction Name], [Begin Time], [End Time],
       [RowLog Contents 0]
FROM fn_dblog(NULL, NULL)
WHERE [Operation] = 'LOP_DELETE_ROWS';
```

La columna RowLog Contents 0 contiene en formato hexadecimal la imagen previa del registro eliminado (Before Image).

### Paso 3: Leer backups del Transaction Log

Si el DELETE ocurrió antes del log activo, se deben restaurar los backups de log en una instancia de recuperación. SQL Server permite restaurar con NORECOVERY para mantener la base en modo de recuperación y luego aplicar logs sucesivos:

```
-- Restaurar backup completo sin recuperar
RESTORE DATABASE ForensicDB
FROM DISK = 'C:\Backups\full_backup.bak'
WITH NORECOVERY, MOVE '...';
-- Aplicar backup de log hasta el momento anterior al DELETE
RESTORE LOG ForensicDB
FROM DISK = 'C:\Backups\log_backup.trn'
WITH STOPAT = '2024-11-15 09:30:00', RECOVERY;
```

### Paso 4: Decodificar la Before Image

La columna RowLog Contents 0 contiene los datos en binario. Para reconstruir el registro se debe mapear el contenido hexadecimal a la estructura de la tabla (columnas y tipos de datos). Este proceso puede hacerse con:

- Herramientas especializadas: ApexSQL Log, Lumigent Log Explorer, SQL Log Rescue
- Scripts T-SQL personalizados usando sys.columns, sys.types y CONVERT()
- Herramientas forenses comerciales: EnCase, FTK con plugins para SQL Server

### Paso 5: Correlacionar con metadatos forenses

Una vez identificada la Before Image, se debe cruzar con información adicional del log para construir la evidencia forense completa:

| Campo del Log | Valor Forense |
|---|---|
| **Transaction ID** | Identifica la transacción exacta donde ocurrió el DELETE |
| **Begin Time / End Time** | Timestamp del borrado (puede diferir del horario local del atacante) |
| **Login Name** | Usuario SQL que ejecutó la operación (correlacionar con sys.dm_exec_sessions) |
| **[RowLog Contents 0]** | Before Image: datos de la fila antes de ser eliminada |
| **[Page ID] + [Slot ID]** | Ubicación física de la fila en la página de datos del .mdf |

Es importante destacar que si el DELETE fue precedido de un UPDATE, ambas operaciones quedarán registradas en el log como transacciones separadas. El análisis forense debe considerar toda la cadena de modificaciones.

### 5. Rol de los Redo Logs y Mecanismo de Recovery

El término Redo Log hace referencia al conjunto de registros del Transaction Log que permiten rehacer (Redo) o deshacer (Undo) operaciones. SQL Server utiliza un proceso llamado **ARIES (Algorithm for Recovery and Isolation Exploiting Semantics)**, que implementa tres fases en la recuperación ante fallos:

| Fase | Función | Relevancia Forense |
|---|---|---|
| Analysis | Determina qué transacciones estaban activas al momento del fallo. | Identifica transacciones sin COMMIT |
| Redo | Rehace todas las operaciones del log desde el último Checkpoint. | Reconstruye el estado previo al fallo |
| Undo | Deshace transacciones que no fueron confirmadas (ROLLBACK). | Permite ver datos de transacciones abortadas |

Desde la perspectiva forense, la fase Undo es particularmente relevante: si un atacante eliminó registros dentro de una transacción que luego fue revertida, tanto el DELETE como el ROLLBACK quedan registrados en el log, preservando evidencia de la intención del borrado.

### 6. Limitaciones y Consideraciones Forenses

- El log activo puede ser truncado si la base está en modelo SIMPLE o si se ejecuta BACKUP LOG WITH TRUNCATE_ONLY.
- Un atacante con permisos de sysadmin puede eliminar backups de log o cambiar el modelo de recuperación.
- La función fn_dblog() solo lee el log activo; para logs históricos se requieren backups o herramientas de terceros.
- Los datos recuperados del log son evidencia digital y deben ser tratados con cadena de custodia adecuada.
- En SQL Server, la eliminación a nivel de página (deallocación) puede ocurrir de forma diferida, lo que puede dejar rastros adicionales en el .mdf para herramientas de carving

# TP 8: Futuro y Zero Trust

**Consigna**

*Investigue y defina el concepto de Privilegios Justo a Tiempo (JIT) dentro de una arquitectura de confianza cero para bases de datos.*

## 1. Introducción: De la Confianza Implícita al Zero Trust

Durante décadas, el modelo de seguridad dominante en entornos corporativos se basó en el paradigma 'castillo y foso': una vez dentro del perímetro de red, los usuarios y sistemas tenían un nivel de confianza implícita. Este enfoque probó ser insuficiente ante amenazas internas, movimiento lateral de atacantes y la adopción de entornos cloud e híbridos.

El modelo Zero Trust (Confianza Cero), formalizado por el analista John Kindervag en 2010 y adoptado por frameworks como NIST SP 800-207, establece un principio fundamental “Nunca confíes, siempre verifica. Asume que la brecha ya ocurrió.”

Dentro de este framework, el acceso privilegiado a bases de datos representa uno de los vectores de mayor riesgo. La solución estructural a este problema es el concepto de Privilegios Justo a Tiempo (JIT Access).

## 2. Definición: Privilegios Justo a Tiempo (JIT Access)

**Just-In-Time Privileged Access (JIT PAM)** es un mecanismo de control de acceso que elimina los permisos permanentes de alto privilegio y los reemplaza por accesos temporales, acotados y auditables que se otorgan únicamente cuando existe una necesidad legítima y verificada, y que expiran automáticamente al finalizar la tarea.

| Modelo Tradicional (Standing Privileges) | Modelo JIT (Zero Trust) |
|---|---|
| **El DBA tiene permisos de sysadmin 24/7** | Los permisos se elevan solo cuando se solicitan y aprueban |
| **Los privilegios persisten aunque no se usen** | Los permisos expiran automáticamente (minutos u horas) |
| **Una cuenta comprometida tiene acceso inmediato** | Una cuenta comprometida sin solicitud activa no tiene privilegios |
| **Auditoría reactiva (post-incidente)** | Auditoría proactiva y en tiempo real de cada elevación |
| **Difícil de escalar en entornos cloud/multi-nube** | Integrado nativamente con plataformas cloud y PAM modernas |

## 3. Arquitectura JIT para Bases de Datos

### 3.1 Componentes Principales

Una implementación JIT para bases de datos se compone de los siguientes elementos:

- Privileged Access Management (PAM): Plataforma central de gestión (CyberArk, BeyondTrust, Azure PIM, HashiCorp Vault). Orquesta solicitudes, aprobaciones y ciclo de vida de las credenciales.
- Identity Provider (IdP): Sistema de identidades (Azure AD, Okta, Active Directory) que autentica al solicitante con MFA antes de cualquier elevación de privilegios.

- Workflow de Aprobación: Motor de reglas que evalúa la solicitud contra políticas definidas (rol del usuario, ventana de tiempo permitida, sistema destino, tipo de operación).
- Credential Broker / Vault: Almacena y rota credenciales de base de datos. Entrega passwords temporales o tokens de sesión sin que el usuario conozca la contraseña real.
- Session Recording: Graba la sesión privilegiada completa (comandos SQL ejecutados, resultados, timestamps) para auditoría y forensia.
- Revocación Automática: Al expirar el tiempo o completar la tarea, los permisos se revocan programáticamente via API o scripts de base de datos.

### 3.2 Flujo Operacional JIT en una Base de Datos

El ciclo de vida de un acceso JIT para un DBA que necesita ejecutar una tarea de mantenimiento en producción sigue estos pasos:

| Paso | Acción | Responsable |
|---|---|---|
| 1. Solicitud | El DBA solicita acceso elevado a través del portal PAM indicando motivo, duración y scope. | DBA / Ingeniero |
| 2. Autenticación MFA | El IdP verifica identidad con segundo factor (TOTP, FIDO2, biometría). | IdP (Azure AD / Okta) |
| 3. Aprobación | El workflow notifica al aprobador. Puede ser automático si cumple políticas pre-aprobadas. | Manager / Policy Engine |
| 4. Provisioning | El PAM otorga el rol necesario en la BD por el tiempo solicitado. Genera credencial temporal. | PAM / Vault |
| 5. Ejecución | El DBA accede a la base de datos. La sesión queda grabada y auditada en tiempo real. | DBA (monitoreado) |
| 6. Revocación | Al expirar el tiempo, los permisos se revocan automáticamente. La sesión queda archivada. | PAM (automático) |

## 4. Reducción del Riesgo en Cuentas de Superusuario

### 4.1 El Problema de los Privilegios Permanentes

Las cuentas de superusuario en bases de datos (sa en SQL Server, SYSDBA en Oracle, superuser en PostgreSQL) representan el mayor riesgo de seguridad por varias razones:

| Amenazas Externas | Amenazas Internas |
|---|---|
| • Credential stuffing / fuerza bruta<br>• Phishing de credenciales DBA<br>• Explotación de vulnerabilidades de autenticación<br>• Supply chain attacks en herramientas DBA | • DBAs con acceso no supervisado a datos sensibles<br>• Uso de credenciales compartidas entre equipos<br>• Empleados desvinculados con acceso residual<br>• Escalamiento de privilegios no autorizado |

### 4.2 Cómo JIT Neutraliza Estos Vectores

La arquitectura JIT reduce la superficie de ataque de cuentas privilegiadas mediante cinco mecanismos fundamentales:

- Eliminación del tiempo de exposición: Un privilegio que no existe no puede ser explotado. La ventana de vulnerabilidad se reduce de 365 días/año a minutos u horas por incidente justificado.
- Principio de mínimo privilegio dinámico: Cada elevación otorga exactamente los permisos necesarios para la tarea específica, no el rol completo de superusuario. Un DBA que necesita ver datos de una tabla no obtiene permiso de DROP DATABASE.
- Segregación de credenciales: El Credential Broker genera contraseñas únicas por sesión, rotadas automáticamente. Un atacante que capture las credenciales de una sesión las encuentra inválidas al intentar reutilizarlas.
- Trazabilidad forzada: Toda elevación de privilegio queda registrada con: identidad verificada por MFA, motivo declarado, aprobación obtenida, comandos ejecutados y tiempo exacto. Imposible negar la acción post-incidente.
- Detección de anomalías en tiempo real: Las plataformas PAM modernas integran UEBA (User and Entity Behavior Analytics) que detectan patrones anómalos durante sesiones privilegiadas y pueden terminar la sesión automáticamente.

## 5. Implementación Práctica en SQL Server / Nube

### 5.1 Azure Privileged Identity Management (PIM) para SQL

En entornos Azure SQL Database o SQL Managed Instance, Azure PIM provee JIT nativo:

```
# Activar rol privilegiado via Azure CLI (JIT)
az role assignment create \
  --assignee 'dba-user@empresa.com' \
  --role 'SQL DB Contributor' \
  --scope '/subscriptions/.../databases/prod-db' \
  --condition "@Request[Action] == 'Microsoft.Sql/servers/read'"
# El rol expira automáticamente según la política PIM configurada
```

### 5.2 JIT con HashiCorp Vault para SQL Server On-Premise

Para entornos on-premise, HashiCorp Vault con el secrets engine de bases de datos provee JIT de credenciales:

```
# Configurar el secrets engine de MSSQL en Vault
vault secrets enable database
vault write database/config/sqlserver \
  plugin_name=mssql-database-plugin \
  connection_url='sqlserver://{{username}}:{{password}}@host:1433' \
  allowed_roles='dba-readonly,dba-maintenance'
# Definir rol JIT con TTL de 1 hora
vault write database/roles/dba-maintenance \
  db_name=sqlserver \
  creation_statements='CREATE LOGIN [{{name}}] WITH PASSWORD=...; ...' \
  default_ttl='1h' max_ttl='4h'
```

## 6. Comparativa de Riesgo: Modelo Tradicional vs. JIT Zero Trust

| Vector de Ataque | Riesgo (Standing Privileges) | Riesgo (JIT Zero Trust) |
|---|---|---|
| Credenciales DBA filtradas | CRÍTICO — Acceso inmediato y total | BAJO — Sin privilegio activo sin aprobación |
| Insider malicioso | ALTO — Acceso no supervisado 24/7 | BAJO — Cada acceso requiere justificación y es grabado |
| Cuenta DBA inactiva | ALTO — Sigue teniendo permisos activos | MÍNIMO — Permisos nunca asignados si no se solicitan |
| Movimiento lateral post-breach | CRÍTICO — Reutilización de credenciales | BAJO — Credenciales de sesión única, no reutilizables |
| Shadow IT / accesos no documentados | ALTO — Difícil de detectar | MÍNIMO — Todo acceso privilegiado pasa por el PAM |

## 7. Marcos de Referencia y Regulaciones que Exigen JIT

La adopción de JIT no es solo una buena práctica: múltiples marcos regulatorios y de seguridad la exigen o recomiendan explícitamente:

- NIST SP 800-207 (Zero Trust Architecture): Define JIT como un control fundamental para el Pilar de Acceso con Mínimo Privilegio.
- CIS Controls v8 — Control 6.8: Establece que los privilegios de administrador deben ser de uso temporal y acotado.
- PCI DSS v4.0 — Requisito 7: Exige que el acceso a datos de tarjetahabiente se restrinja al mínimo necesario y se revise periódicamente.
- SOX (Sarbanes-Oxley) — Sección 404: Requiere controles de acceso sobre sistemas financieros, con evidencia de auditoría de accesos privilegiados.
- ISO/IEC 27001:2022 — Control 5.18: Gestión de derechos de acceso con revisión periódica y principio de mínimo privilegio.

El cumplimiento regulatorio es un driver adicional que impulsa la adopción de JIT en organizaciones que manejan datos sensibles o están sujetas a auditorías externas.